LAB | Hive, Impala, Avro, Parquet & Sqoop (Loudacre)

Week 5 — Day 2: Querying Data Lakes with Apache Hive & Impala

Lab Questions

1. **Q1.** Create an external table in the `default` database on `accounts`.
2. **Q2.** Create an Avro external table on `accounts` avro.
3. **Q3.** Create a Parquet table on `accounts.parquet`.
4. **Q4.** Import directly through Sqoop table called `'basestations'` into Hive.
5. **Q5.** Import only `first_name` and `last_name` using Sqoop in HDFS for table `accounts`. The table should be reflected on Hive/Impala.
6. **Q6.** Run a query on Hive and Impala both: group the accounts on the basis of `city` and order them by their names in alphabetical order.

Note About My Setup (Mac OS) & How I Adapted This Exercise

In the official Cloudera training VM, **MySQL** (`loudacre` database), **Sqoop**, **HDFS**, **Hive**, and **Impala** come pre-installed.

Because I am using a **Mac** and cannot run the Cloudera VM locally, I followed the same approach as in Week 4: 1. First, I wrote the exact **Sqoop, HDFS, Hive (`hive` / `beeline`), and Impala (`impala-shell`)** commands for all 6 questions, along with study notes explaining how each step works. 2. Second, I created a local SQLite database script (create_loudacre_hive.sql) with the `accounts` and `basestations` tables and a bash + Python/PyArrow simulation script (run_hive_impala_lab.sh) that generates the HDFS folders, CSV part files, Avro schema (`accounts.avsc`), real binary Parquet file (`accounts.parquet`), and executes the SQL queries on my Mac.

> **My Quick Notes (To Remember for Class):** - **Internal (Managed) vs. External Table in Hive:** - **Internal (`CREATE TABLE`):** Hive owns both the table metadata and the actual data files in `/user/hive/warehouse/` . If you run `DROP TABLE` , Hive deletes **both** the metadata and the underlying files in HDFS! - **External (`CREATE EXTERNAL TABLE ... LOCATION '...'`):** Hive only manages the table schema/metadata, while the files stay in their original HDFS folder (e.g., `/Loudacre/accounts`). If you run `DROP TABLE` , Hive only removes the table definition—**the data files in HDFS remain safe and untouched**. - **Hive vs. Impala:** - **Hive:** Translates SQL (HiveQL) into MapReduce/Tez/Spark batch jobs. Great for heavy batch transformations and fault-tolerant long queries. - **Impala:** Massively Parallel Processing (MPP) in-memory SQL engine. It reads the exact same HDFS files and shares the **Hive Metastore**, running interactive queries much faster than Hive. - **`INVALIDATE METADATA` in Impala:** Whenever you create a new table in Hive or import a table with `sqoop import --hive-import` , Impala does not automatically know about the change until you run `INVALIDATE METADATA;` (or `INVALIDATE METADATA <table_name>;`) inside `impala-shell` .

Part 1 — Sqoop, HDFS, Hive & Impala Commands for All 6 Questions

Q1. Create an external table in the `default` database on `accounts`

Step 1: Make sure `accounts` is imported into HDFS (from our Sqoop import):

```
sqoop import \
  --connect jdbc:mysql://localhost/loudacre \
  --username training \
  --password training \
  --table accounts \
  --target-dir /Loudacre/accounts
```

Step 2: Open Hive (`hive` or `beeline`) and create the External Table in `default` :

```
USE default;

CREATE EXTERNAL TABLE accounts (
    acct_num INT,
    acct_create_dt STRING,
    acct_close_dt STRING,
    first_name STRING,
    last_name STRING,
    address STRING,
    city STRING,
    state STRING,
    zipcode STRING,
    phone_number STRING,
    created STRING,
    modified STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LOCATION '/Loudacre/accounts';
```

Verify the table in Hive:

```
SHOW TABLES;
SELECT * FROM accounts LIMIT 5;
```

> **My Notes:** - `USE default;` selects Hive's default database. - `EXTERNAL` tells Hive not to move the files into `/user/hive/warehouse/` and not to delete the HDFS files if the table is dropped. - `ROW FORMAT DELIMITED FIELDS TERMINATED BY ','` tells Hive that each column in the Sqoop text file (`part-m-00000`) is separated by a comma. - `LOCATION '/Loudacre/accounts'` points to the HDFS folder containing the files (not an individual file name!).

Q2. Create an Avro external table on `accounts` avro

Step 1: Import `accounts` from MySQL as Avro files using Sqoop:

```
sqoop import \
  --connect jdbc:mysql://localhost/loudacre \
  --username training \
  --password training \
  --table accounts \
  --target-dir /Loudacre/accounts_avro \
  --as-avrodatafile
```

Step 2: Copy the generated `accounts.avsc` schema file into HDFS:

When Sqoop runs with `--as-avrodatafile`, it automatically generates a JSON schema file named `accounts.avsc` in your current local Linux directory. We upload it to HDFS so Hive/Impala can read the schema:

```
hdfs dfs -put accounts.avsc /Loudacre/accounts.avsc
```

Step 3: Create the Avro External Table in Hive:

```
USE default;

CREATE EXTERNAL TABLE accounts_avro
STORED AS AVRO
LOCATION '/Loudacre/accounts_avro'
TBLPROPERTIES ('avro.schema.url'='hdfs:///Loudacre/accounts.avsc');
```

Verify the Avro table:

```
SELECT acct_num, first_name, last_name, city FROM accounts_avro LIMIT 5;
```

> **My Notes:** - `--as-avrodatafile` tells Sqoop to write binary **Avro** files (`part-m-00000.avro`) instead of plain text CSV. - Notice that in `CREATE EXTERNAL TABLE accounts_avro`, we don't even have to type all the column names manually! By setting `TBLPROPERTIES ('avro.schema.url'='hdfs:///Loudacre/accounts.avsc')`, Hive automatically reads the column names and data types directly from the `.avsc` schema file.

Q3. Create a Parquet Table on `accounts.parquet`

Depending on whether `accounts.parquet` is imported via Sqoop/HDFS or created inside Hive/Impala from the existing `accounts` table, here are the two standard ways:

Method A: External Parquet Table pointing to Parquet file(s) in HDFS

1. **If importing with Sqoop as Parquet (or placing `accounts.parquet` in `/Loudacre/accounts_parquet`):** `bash sqoop import \ --connect jdbc:mysql://localhost/loudacre \ --username training \ --password training \ --table accounts \ --target-dir /Loudacre/accounts_parquet \ --as-parquetfile`
2. **Create the Parquet Table in Hive:** ```sql USE default;

CREATE EXTERNAL TABLE accounts_parquet (acct_num INT, acct_create_dt STRING, acct_close_dt STRING, first_name STRING, last_name STRING, address STRING, city STRING, state STRING, zipcode STRING, phone_number STRING, created STRING, modified STRING) STORED AS PARQUET LOCATION '/Loudacre/accounts_parquet'; `` *(Tip: In **Impala**, you can also infer the schema directly from the Parquet file without typing the columns using:`CREATE EXTERNAL TABLE accounts_parquet LIKE PARQUET '/Loudacre/accounts_parquet/accounts.parquet' STORED AS PARQUET LOCATION '/Loudacre/accounts_parquet';`)*

Method B: Create a Parquet Table in Hive directly from our existing `accounts` table (CTAS)

```
USE default;

CREATE TABLE accounts_parquet
STORED AS PARQUET
AS
SELECT * FROM accounts;
```

> **My Notes:** - **Parquet** is a **columnar** binary format (`STORED AS PARQUET`). Unlike CSV or Avro (which store data row by row), Parquet stores values column by column, making analytical queries in Hive, Impala, and Spark much faster and more compressed.

Q4. Import directly through Sqoop table called `'basestations'` into Hive

```
sqoop import \
  --connect jdbc:mysql://localhost/loudacre \
  --username training \
  --password training \
  --table basestations \
  --hive-import
```

Verify inside Hive:

```
SHOW TABLES;
DESCRIBE basestations;
SELECT * FROM basestations LIMIT 10;
```

> **My Notes:** - `--hive-import` tells Sqoop to import the `basestations` table from MySQL directly into **Hive's default warehouse** (`/user/hive/warehouse/basestations`). - Behind the scenes, Sqoop does two things automatically: 1. Copies the data from MySQL into a temporary folder in HDFS. 2. Generates and executes a `CREATE TABLE basestations ...` and `LOAD DATA INPATH ...` statement in Hive! - Optional flags we can add if needed: - `--create-hive-table` : Fails safely if the table `basestations` already exists in Hive. - `--hive-table default.basestations` : Explicitly specifies the database and table name in Hive.

Q5. Import only `first_name` and `last_name` using Sqoop in HDFS for table `accounts`. The table should be reflected on Hive/Impala.

Step 1: Import only `first_name` and `last_name` into HDFS using Sqoop:

```
sqoop import \
  --connect jdbc:mysql://localhost/loudacre \
  --username training \
  --password training \
  --table accounts \
  --columns "first_name,last_name" \
  --target-dir /Loudacre/accounts_names
```

(Note: You can also do Steps 1 & 2 in a single command by replacing `--target-dir /Loudacre/accounts_names` with `--hive-import --hive-table accounts_names`.)

Step 2: Create the External Table in Hive pointing to that HDFS directory:

```
USE default;

CREATE EXTERNAL TABLE accounts_names (
    first_name STRING,
    last_name STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LOCATION '/Loudacre/accounts_names';
```

Step 3: Reflect the table in Impala (`impala-shell`) and verify in both engines:

```
-- Inside impala-shell:
INVALIDATE METADATA accounts_names;

SELECT * FROM accounts_names LIMIT 10;
```

> **My Notes:** - `--columns "first_name,last_name"` tells Sqoop to extract **only** those two columns from the MySQL `accounts` table (Notice: no spaces after the comma inside `"first_name,last_name"`). - Because Hive and Impala share the **Hive Metastore**, creating `accounts_names` in Hive makes it available to Impala—**but** Impala caches metadata in memory for speed. Running `INVALIDATE METADATA accounts_names;` in `impala-shell` refreshes Impala's cache so the table is immediately reflected in both Hive and Impala!

Q6. Run a query on Hive and Impala both: group the accounts on the basis of `city` and order them by their names in alphabetical order.

1. Run in Hive (`hive` CLI):

```
USE default;

-- Query A: Group accounts by city and name, ordered by city and alphabetically by name
SELECT city, first_name, last_name
FROM accounts
GROUP BY city, first_name, last_name
ORDER BY city ASC, first_name ASC, last_name ASC;
```

(Alternative if aggregating all account names into a single row per city in Hive):

```
SELECT
    city,
    COUNT(*) AS total_accounts,
    CONCAT_WS(', ', SORT_ARRAY(COLLECT_LIST(CONCAT(first_name, ' ', last_name)))) AS
ordered_names
FROM accounts
GROUP BY city
ORDER BY city ASC;
```

2. Run in Impala (`impala-shell`):

```
-- First make sure Impala metadata is synced with Hive:
INVALIDATE METADATA accounts;

-- Query A: Group accounts by city and name, ordered by city and alphabetically by name
SELECT city, first_name, last_name
FROM accounts
GROUP BY city, first_name, last_name
ORDER BY city ASC, first_name ASC, last_name ASC;
```

(Alternative if aggregating all account names into a single row per city in Impala):

```
SELECT
    city,
    COUNT(*) AS total_accounts,
    GROUP_CONCAT(CONCAT(first_name, ' ', last_name), ', ') AS account_names
FROM accounts
GROUP BY city
ORDER BY city ASC;
```

> **My Notes:** - In standard SQL (both **Hive** and **Impala**), every non-aggregated column in the `SELECT` list (`city` , `first_name` , `last_name`) must appear in the `GROUP BY` clause. - `ORDER BY city ASC, first_name ASC, last_name ASC` first organises the output by `city` (Austin -> Los Angeles -> Miami -> New York -> Seattle), and inside each city orders the people alphabetically by `first_name` and `last_name` (e.g. inside Los Angeles: *Aaron Carter*, then *Britney Spears*, then *Emma Watson*). - **Execution difference you notice in class:** When you run this query in **Hive**, it launches a MapReduce job (`Map 1: 100%, Reduce 1: 100%`) which takes several seconds because of job startup overhead. When you run the exact same query in **Impala**, it executes in memory and returns the result in less than a second!

Part 2 — My Local Mac Simulation & Terminal Output

To test and verify all 6 questions on macOS without the Cloudera VM, I created: - create_loudacre_hive.sql: Creates the `accounts` and `basestations` tables locally. - run_hive_impala_lab.sh: Simulates the Sqoop imports, creates the Avro schema (`accounts.avsc`) and real Parquet file (`accounts.parquet`) using Python `pandas` / `pyarrow`, and executes the Hive and Impala queries.

Terminal Output from Running `run_hive_impala_lab.sh`:

```
(base) rosi@Calculon IronHack Course  % bash "Week 05/Day 02/run_hive_impala_lab.sh"
============================================================
 Q1: Create an external table in the default database on accounts
============================================================
>>> Step 1: Ensure 'accounts' data is imported into HDFS (/Loudacre/accounts)
$ sqoop import \
  --connect jdbc:mysql://localhost/loudacre \
  --username training \
  --password training \
  --table accounts \
  --target-dir /Loudacre/accounts
INFO mapreduce.ImportJobBase: Retrieved 9 records into /Loudacre/accounts

>>> Step 2: Create the External Table in Hive (default database)
hive> USE default;
OK
hive> CREATE EXTERNAL TABLE accounts (
    >     acct_num INT,
    >     acct_create_dt STRING,
    >     acct_close_dt STRING,
    >     first_name STRING,
    >     last_name STRING,
    >     address STRING,
    >     city STRING,
    >     state STRING,
    >     zipcode STRING,
    >     phone_number STRING,
    >     created STRING,
    >     modified STRING
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > LOCATION '/Loudacre/accounts';
OK
Time taken: 0.245 seconds
hive> SELECT acct_num, first_name, last_name, city, state FROM accounts LIMIT 5;
acct_num  first_name  last_name     city      state
--------  ----------  ---------  ----------  -----
    1001  Britney     Spears     Los Angeles  CA
    1002  Roger       Miller     Austin       TX
    1003  Alice       Johnson    New York     NY
    1004  Carlos      Gomez      Miami        FL
    1005  Emma        Watson     Los Angeles  CA

============================================================
 Q2: Create an Avro external table on accounts avro
============================================================
>>> Step 1: Import 'accounts' from MySQL as Avro using Sqoop (--as-avrodatafile)
$ sqoop import \
  --connect jdbc:mysql://localhost/loudacre \
  --username training \
  --password training \
  --table accounts \
  --target-dir /Loudacre/accounts_avro \
  --as-avrodatafile
INFO orm.CompilationManager: Writing avro schema file: accounts.avsc
INFO mapreduce.ImportJobBase: Retrieved 9 records into /Loudacre/accounts_avro
```

```
>>> Step 2: Copy accounts.avsc schema file to HDFS & Create External Avro Table in Hive
$ hdfs dfs -put accounts.avsc /Loudacre/accounts.avsc

hive> USE default;
hive> CREATE EXTERNAL TABLE accounts_avro
    > STORED AS AVRO
    > LOCATION '/Loudacre/accounts_avro'
    > TBLPROPERTIES ('avro.schema.url'='hdfs:///Loudacre/accounts.avsc');
OK
Time taken: 0.198 seconds
hive> SELECT acct_num, first_name, last_name, city FROM accounts_avro LIMIT 5;
acct_num  first_name  last_name     city
--------  ----------  ---------  -----------
    1001  Britney     Spears     Los Angeles
    1002  Roger       Miller     Austin
    1003  Alice       Johnson    New York
    1004  Carlos      Gomez      Miami
    1005  Emma        Watson     Los Angeles

============================================================
 Q3: Create a Parquet Table on accounts.parquet
============================================================
>>> Step 1: Import 'accounts' as Parquet (or convert via Hive/Impala)
$ sqoop import \
  --connect jdbc:mysql://localhost/loudacre \
  --username training \
  --password training \
  --table accounts \
  --target-dir /Loudacre/accounts_parquet \
  --as-parquetfile
INFO mapreduce.ImportJobBase: Retrieved 9 records into /Loudacre/accounts_parquet

>>> Step 2: Create the Parquet Table in Hive / Impala
hive> USE default;
hive> CREATE EXTERNAL TABLE accounts_parquet (
    >     acct_num INT,
    >     acct_create_dt STRING,
    >     acct_close_dt STRING,
    >     first_name STRING,
    >     last_name STRING,
    >     address STRING,
    >     city STRING,
    >     state STRING,
    >     zipcode STRING,
    >     phone_number STRING,
    >     created STRING,
    >     modified STRING
    > )
    > STORED AS PARQUET
    > LOCATION '/Loudacre/accounts_parquet';
OK
Time taken: 0.212 seconds
hive> SELECT acct_num, first_name, last_name, city, phone_number FROM accounts_parquet
LIMIT 5;
 acct_num first_name last_name        city phone_number
     1001    Britney    Spears Los Angeles     555-0101
     1002      Roger    Miller      Austin     555-0102
     1003      Alice   Johnson    New York     555-0103
     1004     Carlos     Gomez       Miami     555-0104
```

```
     1005       Emma    Watson Los Angeles     555-0105

============================================================
 Q4: Import directly through Sqoop table called 'basestations' into Hive
============================================================
$ sqoop import \
  --connect jdbc:mysql://localhost/loudacre \
  --username training \
  --password training \
  --table basestations \
  --hive-import
INFO mapreduce.ImportJobBase: Retrieved 5 records.
INFO hive.HiveImport: Loading uploaded data into Hive
INFO hive.HiveImport: Hive import complete.

hive> SHOW TABLES LIKE 'basestations';
basestations
hive> SELECT * FROM basestations;
bs_id        address           city      state  zipcode  latitude  longitude
-----  ------------------  ----------  -----  -------  --------  ---------
    1  101 Harbor Way       Los Angeles  CA     90001     34.0522  -118.2437
    2  250 Tech Ridge Pkwy  Austin       TX     73301     30.2672   -97.7431
    3  88 Madison Ave       New York     NY     10118     40.7128    -74.006
    4  15 Ocean Drive       Miami        FL     33132     25.7617   -80.1918
    5  700 Pine St          Seattle      WA     98109     47.6062  -122.3321

============================================================
 Q5: Import only first_name and last_name using Sqoop in HDFS
     for table accounts. Reflect the table on Hive/Impala.
============================================================
>>> Step 1: Import only first_name and last_name into HDFS with Sqoop
$ sqoop import \
  --connect jdbc:mysql://localhost/loudacre \
  --username training \
  --password training \
  --table accounts \
  --columns "first_name,last_name" \
  --target-dir /Loudacre/accounts_names
INFO mapreduce.ImportJobBase: Retrieved 9 records into /Loudacre/accounts_names

>>> Step 2: Check HDFS file content
$ hdfs dfs -cat /Loudacre/accounts_names/part-m-00000
Britney,Spears
Roger,Miller
Alice,Johnson
Carlos,Gomez
Emma,Watson
David,Smith
Aaron,Carter
Tara,Davis
Brian,Adams

>>> Step 3: Create the table in Hive & Refresh Metastore in Impala
hive> CREATE EXTERNAL TABLE accounts_names (
    >     first_name STRING,
    >     last_name STRING
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
```

```
    > LOCATION '/Loudacre/accounts_names';
OK

[localhost:21000] default> INVALIDATE METADATA accounts_names;
Query: invalidate metadata accounts_names
Fetched 0 row(s) in 0.15s

[localhost:21000] default> SELECT * FROM accounts_names;
first_name  last_name
----------  ---------
Britney     Spears
Roger       Miller
Alice       Johnson
Carlos      Gomez
Emma        Watson
David       Smith
Aaron       Carter
Tara        Davis
Brian       Adams

============================================================
 Q6: Run a query on Hive and Impala both: group the accounts
     on the basis of city and order them by their names in
     alphabetical order.
============================================================
>>> 1) Query in Hive (Grouping by city and ordering alphabetically by name):
hive> SELECT city, first_name, last_name
    > FROM accounts
    > GROUP BY city, first_name, last_name
    > ORDER BY city ASC, first_name ASC, last_name ASC;
   city      first_name  last_name
-----------  ----------  ---------
Austin       Roger       Miller
Austin       Tara        Davis
Los Angeles  Aaron       Carter
Los Angeles  Britney     Spears
Los Angeles  Emma        Watson
Miami        Carlos      Gomez
New York     Alice       Johnson
New York     Brian       Adams
Seattle      David       Smith

>>> 2) Same Query + Aggregated City Summary in Impala:
[localhost:21000] default> INVALIDATE METADATA accounts;
[localhost:21000] default> SELECT city, first_name, last_name
                         > FROM accounts
                         > GROUP BY city, first_name, last_name
                         > ORDER BY city ASC, first_name ASC, last_name ASC;
   city      first_name  last_name
-----------  ----------  ---------
Austin       Roger       Miller
Austin       Tara        Davis
Los Angeles  Aaron       Carter
Los Angeles  Britney     Spears
Los Angeles  Emma        Watson
Miami        Carlos      Gomez
New York     Alice       Johnson
New York     Brian       Adams
Seattle      David       Smith
```

```
[localhost:21000] default> SELECT city, COUNT(*) AS total_accounts,
                         >        GROUP_CONCAT(first_name || ' ' || last_name, ', ') AS
ordered_names
                         > FROM (SELECT city, first_name, last_name FROM accounts ORDER BY
city, first_name, last_name) t
                         > GROUP BY city
                         > ORDER BY city ASC;
   city      total_accounts               ordered_names
-----------  --------------  ----------------------------------------
Austin                    2  Roger Miller, Tara Davis
Los Angeles               3  Aaron Carter, Britney Spears, Emma Watson
Miami                     1  Carlos Gomez
New York                  2  Alice Johnson, Brian Adams
Seattle                   1  David Smith
============================================================
```